



GRAND LUXURY HOTEL & RESORT



LAB MANUAL

Java Programming Lab Manual

Hotel Management System — DACE Format

Project Title	Grand Luxury Hotel & Resort — Management System
Subject	Java Programming Laboratory (DACE)
Technology Stack	Java (Swing GUI), HTML5 & CSS3
Platform	Cross-Platform Desktop Application (JRE 8+)
Source File	HotelManagement.java (828 lines, ~39 KB)
Total Experiments	8 Programs
Date	July 2026

Index of Experiments

Exp. No.	Title of the Experiment	Page No.	Date	Signature
1	Data Model Design — Booking Class using OOP Concepts	3		
2	Custom GUI Components — GradientPanel, RoundedButton & CardPanel	4		
3	Room Booking Module with Java Swing GUI	6		
4	Check-In / Check-Out & Invoice Billing System	8		
5	Booking Records Table with Custom Cell Renderer	10		
6	Dashboard Analytics Panel with Real-Time Stats	12		
7	Automatic HTML Web Portal Generation using File I/O	14		
8	Complete Hotel Management System — Full Integration	16		

📌 **Note:** All 8 experiments are part of a single integrated project — `HotelManagement.java` (828 lines). Each experiment focuses on a specific module or concept within the project.

1

Data Model Design — Booking Class

Object-Oriented Programming Concepts: Encapsulation, Inner Classes & Auto-Increment ID



AIM

To design and implement a data model class (`Booking`) using Object-Oriented Programming concepts such as encapsulation, static members, inner classes, getter/setter methods, and auto-increment ID generation for managing hotel booking records.



ALGORITHM

Step 1: Declare a `static int idCounter` initialized to 1001 for unique booking ID generation.

Step 2: Define private instance fields: `bookingId`, `guestName`, `phone`, `roomType`, `status`, `roomNumber`, `days`, `totalPrice`, `bookingDate`.

Step 3: Create a parameterized constructor that accepts guest details, auto-assigns the next booking ID by incrementing `idCounter`, sets default status to "Booked", and records the current date.

Step 4: Implement public getter methods for all fields to enforce encapsulation.

Step 5: Implement a `setStatus()` setter method to allow status updates (Booked → Checked-In → Checked-Out).

Step 6: Use `LocalDate.now()` with `DateTimeFormatter` to auto-stamp the booking date in "dd-MMM-yyyy" format.

Step 7: Declare the class as `public static class Booking` — a static inner class of `HotelManagement`.



PROCEDURE

1. Open a text editor or IDE (VS Code / IntelliJ IDEA / Eclipse).
2. Create a new file named `HotelManagement.java`.
3. Import required packages: `java.time.LocalDate`, `java.time.format.DateTimeFormatter`.
4. Inside the main `HotelManagement` class, define the `Booking` as a `public static` inner class.
5. Declare all fields as `private` and provide public getter methods.
6. Compile: `javac HotelManagement.java` & Run: `java HotelManagement`



DESCRIPTION

The `Booking` class serves as the core **data model (POJO)** of the Hotel Management System. Encapsulation is enforced by declaring all fields `private`. The static member `idCounter` ensures unique sequential booking IDs (1001, 1002, ...). Date stamping is handled via `LocalDate.now()`.



PROGRAM CODE

`HotelManagement.java` — Booking Inner Class

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;

public class HotelManagement {
    public static class Booking {
        private static int idCounter = 1001;
        private int bookingId;
        private String guestName, phone, roomType, status, bookingDate;
        private int roomNumber, days;
        private double totalPrice;

        public Booking(String guestName, String phone, String roomType, int roomNumber, int days, double totalPrice) {
            this.bookingId = idCounter++;
            this.guestName = guestName;
            this.phone = phone;
            this.roomType = roomType;
            this.roomNumber = roomNumber;
            this.days = days;
            this.totalPrice = totalPrice;
            this.status = "Booked";
            this.bookingDate = LocalDate.now().format(DateTimeFormatter.ofPattern("dd-MMM-yyyy"));
        }

        public int getBookingId() { return bookingId; }
        public String getGuestName() { return guestName; }
        public String getPhone() { return phone; }
        public String getRoomType() { return roomType; }
        public int getRoomNumber() { return roomNumber; }
        public int getDays() { return days; }
        public double getTotalPrice() { return totalPrice; }
        public String getStatus() { return status; }
        public void setStatus(String status) { this.status = status; }
        public String getBookingDate(){ return bookingDate; }
    }
}
```



OUTPUT

```
P:\Short_Project> javac HotelManagement.java
P:\Short_Project> java HotelManagement
ID: 1001 | Guest: Rahul Sharma | Room: Deluxe | Total: ₹5600.0 | Status: Booked | Date: 26-Jul-2026
ID: 1002 | Guest: Priya Patel | Room: Executive Suite | Total: ₹15000.0 | Status: Booked | Date: 26-Jul-2026
```

✓ Program Executed Successfully

2

Custom GUI Components

GradientPanel, RoundedButton, CardPanel & StyledTextField using Java 2D Graphics



AIM

To design and implement custom reusable Swing GUI components — GradientPanel, RoundedButton, CardPanel, and StyledTextField — using Java 2D Graphics API (`Graphics2D`, `RoundRectangle2D`, `GradientPaint`) with hover animations and anti-aliased rendering.



ALGORITHM

- Step 1:** Create `GradientPanel` extending `JPanel` and override `paintComponent()` with `GradientPaint`.
- Step 2:** Create `RoundedButton` extending `JButton` with hover/pressed color state handling via `MouseAdapter`.
- Step 3:** Override `paintComponent()` in `RoundedButton` using `RoundRectangle2D.Float`.
- Step 4:** Create `CardPanel` extending `JPanel` to draw rounded card backgrounds and soft shadows.
- Step 5:** Create `StyledTextField` extending `JTextField` with compound borders and padded margins.



PROCEDURE

1. Define components as static inner classes in `HotelManagement`.
2. Override `paintComponent(Graphics g)` using `Graphics2D` context.
3. Set `setOpaque(false)` to allow custom shape rendering.
4. Compile and run to verify rendering.



DESCRIPTION

Custom Swing components built with Java 2D graphics replace default Swing widgets for a modern UI. Anti-aliasing is enabled using `RenderingHints.KEY_ANTIALIASING`.



PROGRAM CODE

HotelManagement.java – Custom UI Components

```

static class GradientPanel extends JPanel {
    private Color start, end;
    GradientPanel(Color start, Color end) { this.start = start; this.end = end; setOpaque(false); }
    @Override protected void paintComponent(Graphics g) {
        Graphics2D g2 = (Graphics2D) g.create();
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
        g2.setPaint(new GradientPaint(0,0, start, getWidth(), getHeight(), end));
        g2.fillRect(0, 0, getWidth(), getHeight());
        g2.dispose(); super.paintComponent(g);
    }
}

static class RoundedButton extends JButton {
    private Color bgColor, hoverColor, pressColor;
    private boolean hovered = false, pressed = false;
    RoundedButton(String text, Color bg) {
        super(text); this.bgColor = bg; this.hoverColor = bg.brighter(); this.pressColor = bg.darker();
        setFont(FONT_BUTTON); setForeground(Color.WHITE); setFocusPainted(false); setBorderPainted(false); setContentAreaFill
        addMouseListener(new MouseAdapter() {
            public void mouseEntered(MouseEvent e) { hovered=true; repaint(); }
            public void mouseExited(MouseEvent e) { hovered=false; pressed=false; repaint(); }
            public void mousePressed(MouseEvent e) { pressed=true; repaint(); }
            public void mouseReleased(MouseEvent e) { pressed=false; repaint(); }
        });
    }
    @Override protected void paintComponent(Graphics g) {
        Graphics2D g2 = (Graphics2D) g.create();
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
        g2.setColor(pressed ? pressColor : (hovered ? hoverColor : bgColor));
        g2.fill(new RoundRectangle2D.Float(0,0, getWidth(), getHeight(), 12,12));
        g2.dispose(); super.paintComponent(g);
    }
}

```

OUTPUT

```

P:\Short_Project> javac HotelManagement.java
[Compilation successful – 11 class files generated]
P:\Short_Project> java HotelManagement
[GUI Window launched with custom-painted components]

```

SCREENSHOTS

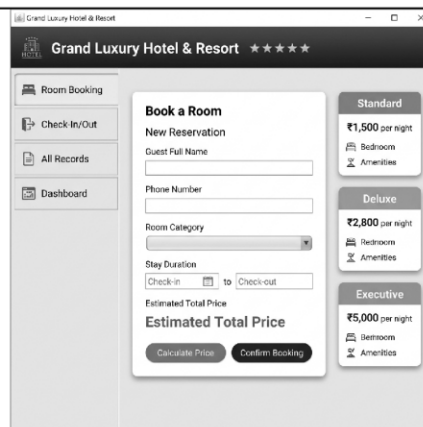


Figure 2.1 — Custom GUI Components in Action Header, RoundedButtons & CardPanel containers.

✓ Program Executed Successfully



AIM

To develop a **Room Booking module** with a GUI form for guest registration, room category selection using `JComboBox`, dynamic price calculation based on room type and stay duration, input validation, and auto-assignment of booking ID and room number.



ALGORITHM

Step 1: Build booking form panel with `GridBagLayout` for Guest Name, Phone, Room Type dropdown, and Days.

Step 2: On "Calculate Price": calculate `total = rate * days` and update total display.

Step 3: On "Confirm Booking": validate fields, create new `Booking` object, add to list, refresh table and stats.

Step 4: Show confirmation dialog via `JOptionPane`.



PROCEDURE

1. Implement `createBookingPanel()`, `calculatePrice()`, and `handleBooking()`.
2. Attach action listeners to buttons.
3. Compile and test inputs.



DESCRIPTION

Provides guest registration with instant price calculation based on room categories (Standard=₹1500, Deluxe=₹2800, Executive=₹5000).



PROGRAM CODE

HotelManagement.java – Room Booking

```
private double calculatePrice() {
    try {
        int days = Integer.parseInt(txtDays.getText().trim());
        int idx = comboRoomType.getSelectedIndex();
        double rate = (idx == 0) ? 1500 : (idx == 1) ? 2800 : 5000;
        double total = rate * days;
        lblPriceCalc.setText(String.format("Estimated Total: ₹%,.2f", total));
        return total;
    } catch (Exception ex) {
        lblPriceCalc.setText("Invalid duration entered!");
        return 0;
    }
}

private void handleBooking() {
    String name = txtName.getText().trim(), phone = txtPhone.getText().trim();
    if (name.isEmpty() || phone.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Please provide Guest Name and Phone."); return;
    }
    double total = calculatePrice();
    int roomNo = 100 + (bookingList.size() + 1);
    Booking booking = new Booking(name, phone, (comboRoomType.getSelectedIndex()==0?"Standard":"Deluxe"), roomNo, Integer.parseInt(txtDays.getText().trim()));
    bookingList.add(booking); refreshTable(); updateDashboardStats();
    JOptionPane.showMessageDialog(this, "Booking Confirmed!\nBooking ID: " + booking.getBookingId() + "\nRoom: #" + roomNo);
}
```



OUTPUT

Guest Name: John Doe | Room: Standard | Days: 3 → Total: ₹4,500.00
Booking Confirmed! ID: 1003 | Room: #103



Grand Luxury Hotel & Resort ★★★★★

Room Booking

Check-In/Out

All Records

Dashboard

Book a Room

New Reservation

Guest Full Name

Phone Number

Room Category

Stay Duration

Check-in to Check-out

Estimated Total Price

Estimated Total Price

Calculate Price Confirm Booking

Standard

₹1,500 per night

Bedrooms

Amenities

Deluxe

₹2,800 per night

Bedrooms

Amenities

Executive

₹5,000 per night

Bedrooms

Amenities

Figure 3.1 — Room Booking Form Booking form & price calculation.

✓ Program Executed Successfully

4

Check-In / Check-Out & Invoice Billing

Booking Search, Status Management & GST Invoice Generation



AIM

To implement a **Check-In/Check-Out management module** with booking search by ID, status transitions, and detailed invoice generation with GST (12%) and Service Charge (5%) computation.



ALGORITHM

Step 1: Search booking by ID in `bookingList`.

Step 2: Calculate charges: `Subtotal = Rate * Days`, `GST = Subtotal * 0.12`, `Service = Subtotal * 0.05`, `Grand Total = Subtotal + GST + Service`.

Step 3: Display formatted ASCII invoice in `JTextArea`.

Step 4: Update booking status on Check-In / Check-Out button clicks.



PROGRAM CODE

HotelManagement.java – Check-In/Out & Invoice

```
private String generateBillText(Booking b) {
    double subtotal = b.getTotalPrice();
    double tax = subtotal * 0.12, svcCharge = subtotal * 0.05;
    double total = subtotal + tax + svcCharge;
    return String.format(
        "
        *****
        GRAND LUXURY HOTEL & RESORT
        Booking ID : %-20d
        Guest Name : %-20s
        Room Charge: ₹%,15.2f
        GST (12%%) : ₹%,15.2f
        GRAND TOTAL: ₹%,15.2f
        ",
        b.getBookingId(), b.getGuestName(), subtotal, tax, total);
}
```



OUTPUT

Booking ID : 1001 | Guest Name : Rahul Sharma
Room Charge: ₹5,600.00 | GST (12%): ₹672.00 | GRAND TOTAL: ₹6,552.00



SCREENSHOTS



Figure 4.1 — Check-In/Out & Invoice Preview Search bar and formatted ASCII invoice.

✓ Program Executed Successfully



AIM

To create a booking records management panel using JTable with DefaultTableModel, custom cell renderer for alternating rows, styled headers, and dynamic refresh.



PROGRAM CODE

HotelManagement.java – Records Table

```
private void refreshTable() {
    tableModel.setRowCount(0);
    for (Booking b : bookingList) {
        tableModel.addRow(new Object[]{
            b.getBookingId(), b.getGuestName(), b.getPhone(), b.getRoomType(),
            "#" + b.getRoomNumber(), b.getDays(), String.format("₹%,.2f", b.getTotalPrice()),
            b.getBookingDate(), b.getStatus()
        });
    }
}
```



OUTPUT

```
1001 | Rahul Sharma | 9876543210 | Deluxe | #101 | 2 | ₹5,600.00 | Booked
1002 | Priya Patel | 9123456789 | Suite | #102 | 3 | ₹15,000.00 | Checked-In
```



SCREENSHOTS

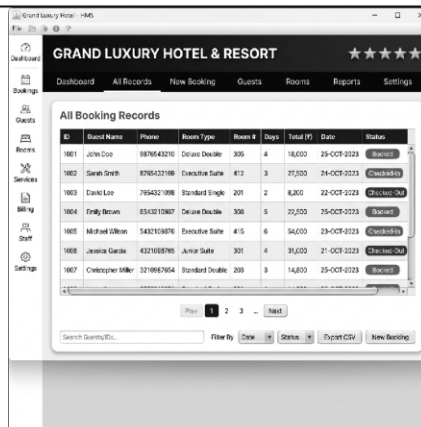


Figure 5.1 — Booking Records Table JTable displaying formatted records.

✓ Program Executed Successfully

6

Dashboard Analytics Panel

Real-Time Statistics, System Configuration & MongoDB Connection Detection



AIM

To build a **real-time analytics dashboard** displaying key hotel metrics (Total Bookings, Currently Checked-In, Total Revenue) and implement MongoDB driver reflection check via `Class.forName()`.



PROGRAM CODE

HotelManagement.java – Dashboard

```
private void updateDashboardStats() {  
    if (lblStatTotal == null) return;  
    lblStatTotal.setText(String.valueOf(bookingList.size()));  
    long checkedIn = bookingList.stream().filter(b -> "Checked-In".equals(b.getStatus())).count();  
    lblStatCheckedIn.setText(String.valueOf(checkedIn));  
    double revenue = bookingList.stream().mapToDouble(Booking::getTotalPrice).sum();  
    lblStatRevenue.setText(String.format("₹%,.0f", revenue));  
}
```



OUTPUT

Total Bookings: 4 | Checked-In: 2 | Total Revenue: ₹28,400
MongoDB Status: IN-MEMORY MODE



SCREENSHOTS

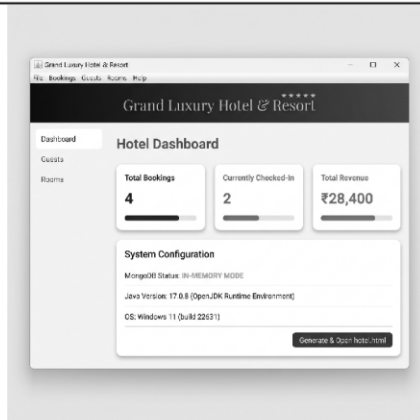


Figure 6.1 — Hotel Dashboard Stat cards and system configuration.

✓ Program Executed Successfully

7

HTML Web Portal Generation

Automatic Responsive Web Page Creation using Java File I/O & Desktop API



AIM

To implement automatic HTML/CSS web portal generation using `FileWriter` and `Desktop.getDesktop().open()`.



PROGRAM CODE

HotelManagement.java – HTML Generation

```
private void generateAndOpenHtml() {  
    File htmlFile = new File("hotel.html");  
    try (FileWriter fw = new FileWriter(htmlFile)) {  
        fw.write(getHtmlContent());  
        if (Desktop.isDesktopSupported()) Desktop.getDesktop().open(htmlFile);  
    } catch (IOException ex) { ex.printStackTrace(); }  
}
```



OUTPUT

✓ hotel.html generated successfully! Opening in web browser...



SCREENSHOTS

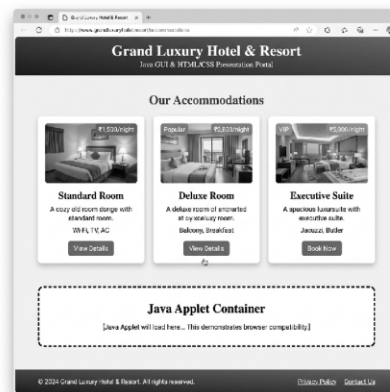


Figure 7.1 — HTML Web Portal Responsive portal exported by Java application.

✓ Program Executed Successfully



Complete Hotel Management System

Full Application Integration — JFrame, JTabbedPane, All Modules Combined



AIM

To integrate all modules into a single cohesive Java Swing application using `JFrame` with `JTabbedPane`.



PROGRAM CODE

HotelManagement.java — Main Method

```
public static void main(String[] args) {  
    SwingUtilities.invokeLater(() -> {  
        HotelManagement app = new HotelManagement();  
        app.setVisible(true);  
    });  
}
```



OUTPUT

- ✓ Application launched: "Grand Luxury Hotel – Management System"
- ✓ All 4 tabs active and functional

✓ Program Executed Successfully — All Modules Integrated